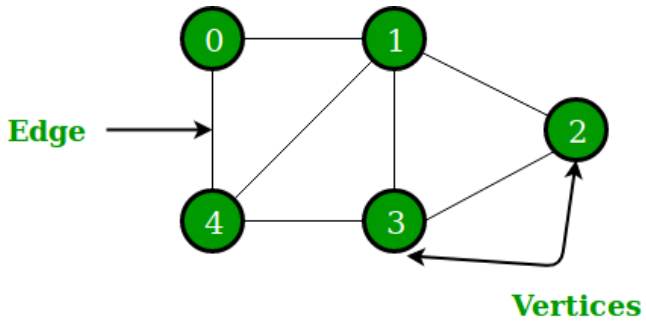
UNIT – V

GRAPH

A Graph is a non-linear data structure consisting of vertices and edges. The vertices are sometimes also referred to as nodes and the edges are lines or arcs that connect any two nodes in the graph. More formally a Graph is composed of a set of vertices(V) and a set of edges(E). The graph is denoted by G (E, V).



Components of a Graph-

Vertices: Vertices are the fundamental units of the graph. Sometimes, vertices are also known as vertex or nodes. Every node/vertex can be labeled or unlabeled.

Edges: Edges are drawn or used to connect two nodes of the graph. It can be ordered pair of nodes in a directed graph. Edges can connect any two nodes in any possible way. There are no rules. Sometimes, edges are also known as arcs. Every edge can be labeled/unlabeled.

Representations of Graph-

Here are the two most common ways to represent a graph :

- Adjacency Matrix
- Adjacency List

1. Adjacency Matrix

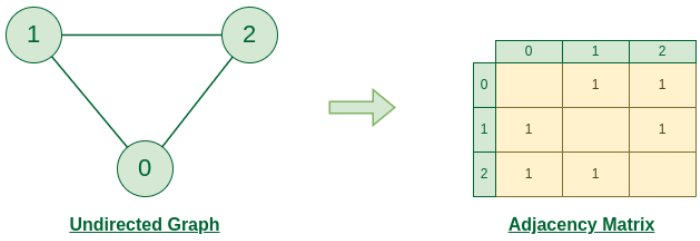
An adjacency matrix is a way of representing a graph as a matrix of boolean (0's and 1's).

Let's assume there are n vertices in the graph So, create a 2D matrix adjMat[n][n] having dimension n x n.

- If there is an edge from vertex i to j, mark adjMat[i][j] as 1.
- If there is no edge from vertex i to j, mark adjMat[i][j] as 0.

Representation of Undirected Graph to Adjacency Matrix:

The below figure shows an undirected graph. Initially, the entire Matrix is initialized to 0. If there is an edge from source to destination, we insert 1 to both cases (adjMat[destination] and adjMat[destination]) because we can go either way.

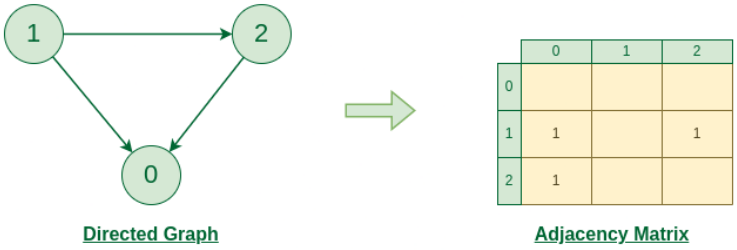


Undirected Graph Adjacency Matrix

Graph Representation of Undirected graph to Adjacency Matrix

Representation of Directed Graph to Adjacency Matrix:

The below figure shows a directed graph. Initially, the entire Matrix is initialized to 0. If there is an edge from source to destination, we insert 1 for that particular adjMat[destination].



Directed Graph Adjacency Matrix

Graph Representation of Directed graph to Adjacency Matrix

2. Adjacency List

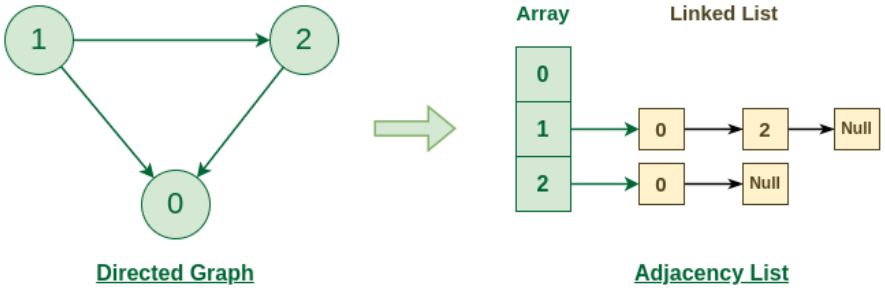
An array of Lists is used to store edges between two vertices. The size of array is equal to the number of vertices (i.e, n). Each index in this array represents a specific vertex in the graph. The entry at the index i of the array contains a linked list containing the vertices that are adjacent to vertex i.

Let's assume there are n vertices in the graph So, create an array of list of size n as adjList[n].

- adjList[0] will have all the nodes which are connected (neighbour) to vertex 0.
- adjList[1] will have all the nodes which are connected (neighbour) to vertex 1 and so on.

Representation of Directed Graph to Adjacency list:

The below directed graph has 3 vertices. So, an array of list will be created of size 3, where each indices represent the vertices. Now, vertex 0 has no neighbours. For vertex 1, it has two neighbour (i.e, 0 and 2) So, insert vertices 0 and 2 at indices 1 of array. Similarly, for vertex 2, insert its neighbours in array of list.

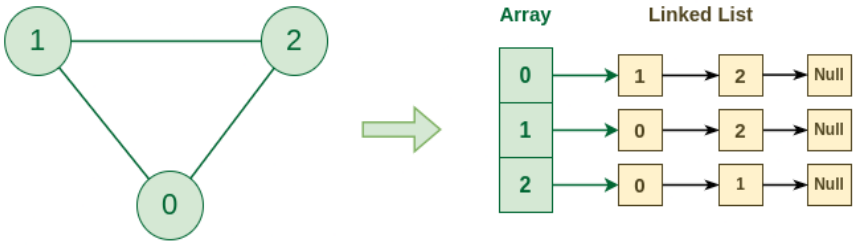


Directed Graph Adjacency List

Graph Representation of Directed graph to Adjacency List

Representation of Undirected Graph to Adjacency list:

The undirected graph below has 3 vertices. So, an array of lists will be created of size 3, where each indices represent the vertices. Now, vertex 0 has two neighbours (i.e, 1 and 2). So, insert vertex 1 and 2 at indices 0 of array. Similarly, for vertex 1, it has two neighbour (i.e, 2 and 1) So, insert vertices 2 and 1 at indices 1 of array. Similarly, for vertex 2, insert its neighbours in array of list.



Undirected Graph Adjacency List

Graph Representation of Undirected graph to Adjacency List

All Operation on Graph-

Graphs are versatile data structures used to model relationships between entities. Here are some common operations that can be performed on graphs:

1. **Add Vertex:**
   Add a new vertex to the graph.

2. **Add Edge:**
   Add an edge between two vertices, possibly with a weight.

3. **Remove Vertex:**
   Remove a vertex and all associated edges from the graph.

4. **Remove Edge:**
   Remove an edge between two vertices.

5. **Check if Vertex Exists:**
   Check if a vertex is present in the graph.

6. **Check if Edge Exists:**
   Check if an edge exists between two vertices.

7. **Get Neighbors of a Vertex:**
   Retrieve all vertices directly connected to a given vertex.

8. **Get All Vertices:**
   Retrieve a list of all vertices in the graph.

9. **Get All Edges:**
   Retrieve a list of all edges in the graph.

10. **Calculate Degree of a Vertex:**
    Determine the number of edges incident to a vertex (in-degree and out-degree for directed graphs).

11. **Breadth-First Search (BFS):**
    Traverse the graph level by level starting from a specified vertex.

12. **Depth-First Search (DFS):**
    Traverse the graph by exploring as far as possible along each branch before backtracking.

13. **Topological Sorting:**
Order the vertices of a directed acyclic graph (DAG) such that for every directed edge u -> v, vertex u comes before v.

14. **Shortest Path Algorithms:**
Find the shortest path between two vertices, commonly using Dijkstra's algorithm or Bellman-Ford algorithm.

15. **Minimum Spanning Tree (MST):**
Find the minimum spanning tree of a connected, undirected graph.

16. **Connectivity Check:**
Check if the graph is connected or find connected components.

17. **Cycle Detection:**
Determine if the graph contains cycles.

18. **Graph Traversals:**
Explore the entire graph by visiting its vertices and edges.

19. **Graph Representation:**
Choose an appropriate representation such as an adjacency matrix, adjacency list, or an edge list.

20. **Graph Coloring:**
Assign colors to vertices in such a way that no two adjacent vertices have the same color.

Below is Single program on all operation of graph:

```
import java.util.*;

class Graph {
   private Map<Integer, List<Integer>> adjacencyList;

   public Graph() {
      this.adjacencyList = new HashMap<>();
   }

   public void addVertex(int vertex) {
      adjacencyList.put(vertex, new ArrayList<>());
   }

   public void addEdge(int source, int destination) {
      adjacencyList.get(source).add(destination);
      adjacencyList.get(destination).add(source); // For undirected graph
   }

   public void removeVertex(int vertex) {
      adjacencyList.remove(vertex);
      for (List<Integer> edges : adjacencyList.values()) {
         edges.remove(Integer.valueOf(vertex));
      }
   }

   public void removeEdge(int source, int destination) {
      adjacencyList.get(source).remove(Integer.valueOf(destination));
      adjacencyList.get(destination).remove(Integer.valueOf(source)); // For undirected graph
   }

   public boolean containsVertex(int vertex) {
      return adjacencyList.containsKey(vertex);
   }

   public boolean containsEdge(int source, int destination) {
      return adjacencyList.get(source).contains(destination);
   }

   public List<Integer> getNeighbors(int vertex) {
      return adjacencyList.get(vertex);
```

```
}

public Set<Integer> getAllVertices() {
    return adjacencyList.keySet();
}

public List<int[]> getAllEdges() {
    List<int[]> edges = new ArrayList<>();
    for (int source : adjacencyList.keySet()) {
        for (int destination : adjacencyList.get(source)) {
            edges.add(new int[]{source, destination});
        }
    }
    return edges;
}

public int getDegree(int vertex) {
    return adjacencyList.get(vertex).size();
}

public void bfs(int start) {
    Set<Integer> visited = new HashSet<>();
    Queue<Integer> queue = new LinkedList<>();

    visited.add(start);
    queue.add(start);

    while (!queue.isEmpty()) {
        int current = queue.poll();
        System.out.print(current + " ");

        for (int neighbor : adjacencyList.get(current)) {
            if (!visited.contains(neighbor)) {
                visited.add(neighbor);
                queue.add(neighbor);
            }
        }
    }
}

public void dfs(int start) {
```

```
        Set<Integer> visited = new HashSet<>();
        dfsRecursive(start, visited);
    }

    private void dfsRecursive(int current, Set<Integer> visited) {
        visited.add(current);
        System.out.print(current + " ");

        for (int neighbor : adjacencyList.get(current)) {
            if (!visited.contains(neighbor)) {
                dfsRecursive(neighbor, visited);
            }
        }
    }

    public void displayGraphDetails() {
        System.out.println("Vertices: " + getAllVertices());
        System.out.println("Edges: " + getAllEdges());
    }
}

public class GraphOperations {
    public static void main(String[] args) {
        Graph graph = new Graph();

        // Adding vertices
        graph.addVertex(1);
        graph.addVertex(2);
        graph.addVertex(3);
        graph.addVertex(4);

        // Adding edges
        graph.addEdge(1, 2);
        graph.addEdge(2, 3);
        graph.addEdge(3, 4);
        graph.addEdge(4, 1);

        // Displaying graph details
        System.out.println("Initial Graph Details:");
        graph.displayGraphDetails();
```

```
        // Performing BFS traversal
        System.out.println("\nBFS Traversal starting from vertex 1:");
        graph.bfs(1);

        // Performing DFS traversal
        System.out.println("\n\nDFS Traversal starting from vertex 1:");
        graph.dfs(1);

        // Removing an edge
        graph.removeEdge(1, 2);

        // Displaying updated graph details
        System.out.println("\n\nGraph Details after removing an edge:");
        graph.displayGraphDetails();

        // Checking existence
        System.out.println("\nVertex 2 exists: " + graph.containsVertex(2));
        System.out.println("Edge between 3 and 4 exists: " + graph.containsEdge(3, 4));

        // Calculating degree
        System.out.println("\nDegree of vertex 3: " + graph.getDegree(3));

        // Removing a vertex
        graph.removeVertex(3);

        // Displaying final graph details
        System.out.println("\nGraph Details after removing vertex 3:");
        graph.displayGraphDetails();
    }
}
```

Shortest path algorithm-

Dijkstra's algorithm is a greedy algorithm used to find the shortest path between a source vertex and all other vertices in a weighted graph with non-negative edge weights. It was conceived by computer scientist Edsger W. Dijkstra and is widely used in various applications, including computer networking and routing protocols.

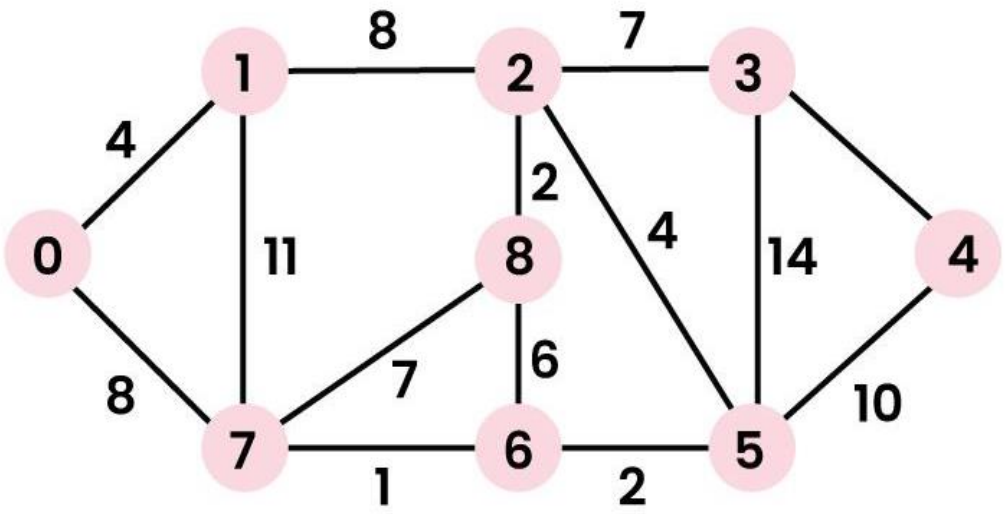
How to find Shortest Paths from Source to all Vertices using Dijkstra's Algorithm

Given a weighted graph and a source vertex in the graph, find the shortest paths from the source to all the other vertices in the given graph.

Note: The given graph does not contain any negative edge.

Examples:

Input: *src = 0, the graph is shown below.*



Output: *0 4 12 19 21 11 9 8 14*

Explanation: *The distance from 0 to 1 = 4.*
The minimum distance from 0 to 2 = 12. 0->1->2
The minimum distance from 0 to 3 = 19. 0->1->2->3
The minimum distance from 0 to 4 = 21. 0->7->6->5->4
The minimum distance from 0 to 5 = 11. 0->7->6->5
The minimum distance from 0 to 6 = 9. 0->7->6
The minimum distance from 0 to 7 = 8. 0->7
The minimum distance from 0 to 8 = 14. 0->1->2->8

Dijkstra's Algorithm using Adjacency Matrix:

The idea is to generate a SPT (shortest path tree) with a given source as a root. Maintain an Adjacency Matrix with two sets,

- one set contains vertices included in the shortest-path tree,
- other set includes vertices not yet included in the shortest-path tree.

At every step of the algorithm, find a vertex that is in the other set (set not yet included) and has a minimum distance from the source.

Algorithm:

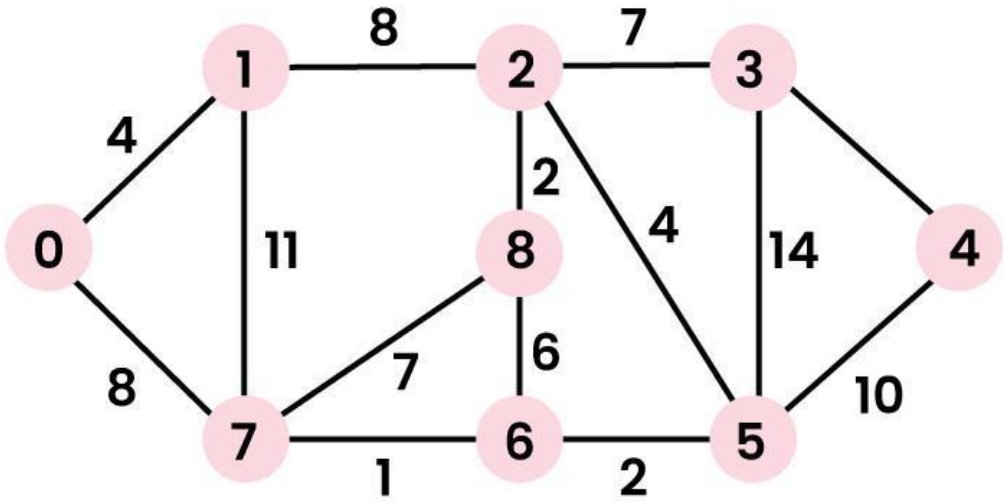
- Create a set sptSet (shortest path tree set) that keeps track of vertices included in the shortest path tree, i.e., whose minimum distance from the source is calculated and finalized. Initially, this set is empty.
- Assign a distance value to all vertices in the input graph. Initialize all distance values as INFINITE. Assign the distance value as 0 for the source vertex so that it is picked first.
- While sptSet doesn't include all vertices
- Pick a vertex u that is not there in sptSet and has a minimum distance value.
- Include u to sptSet.
- Then update the distance value of all adjacent vertices of u.
- To update the distance values, iterate through all adjacent vertices.
- For every adjacent vertex v, if the sum of the distance value of u (from source) and weight of edge u-v, is less than the distance value of v, then update the distance value of v.

Note: We use a boolean array sptSet[] to represent the set of vertices included in SPT. If a value sptSet[v] is true, then vertex v is included in SPT, otherwise not. Array dist[] is used to store the shortest distance values of all vertices.

Illustration of Dijkstra Algorithm:

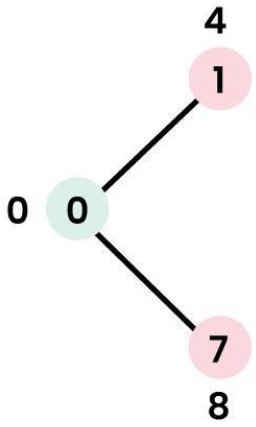
To understand the Dijkstra's Algorithm lets take a graph and find the shortest path from source to all nodes.

Consider below graph and src = 0



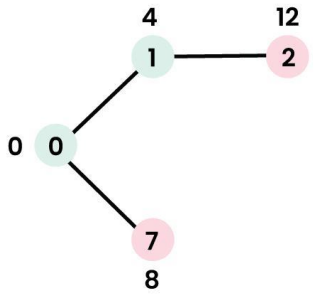
Step 1:

- The set sptSet is initially empty and distances assigned to vertices are {0, INF, INF, INF, INF, INF, INF, INF} where INF indicates infinite.
- Now pick the vertex with a minimum distance value. The vertex 0 is picked, include it in sptSet. So sptSet becomes {0}. After including 0 to sptSet, update distance values of its adjacent vertices.
- Adjacent vertices of 0 are 1 and 7. The distance values of 1 and 7 are updated as 4 and 8.
- The following subgraph shows vertices and their distance values, only the vertices with finite distance values are shown. The vertices included in SPT are shown in green colour.



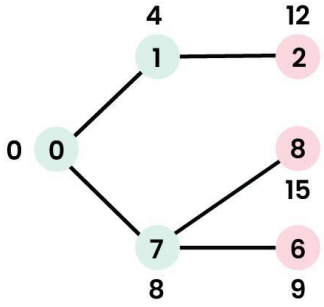
Step 2:

- Pick the vertex with minimum distance value and not already included in SPT (not in sptSET). The vertex 1 is picked and added to sptSet.
- So sptSet now becomes {0, 1}. Update the distance values of adjacent vertices of 1.
- The distance value of vertex 2 becomes 12.



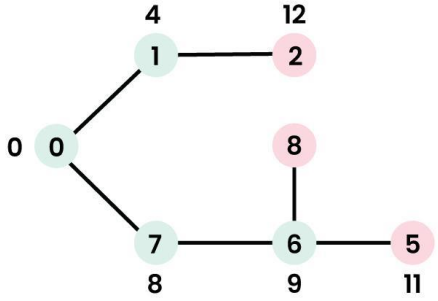
Step 3:

- Pick the vertex with minimum distance value and not already included in SPT (not in sptSET). Vertex 7 is picked. So sptSet now becomes {0, 1, 7}.
- Update the distance values of adjacent vertices of 7. The distance value of vertex 6 and 8 becomes finite (15 and 9 respectively).

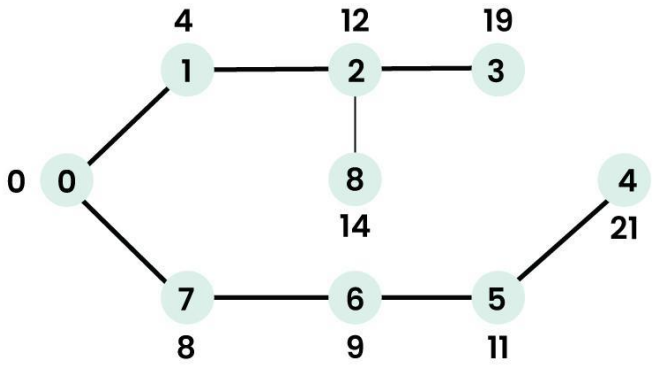


Step 4:

- Pick the vertex with minimum distance value and not already included in SPT (not in sptSET). Vertex 6 is picked. So sptSet now becomes {0, 1, 7, 6}.
- Update the distance values of adjacent vertices of 6. The distance value of vertex 5 and 8 are updated.



We repeat the above steps until sptSet includes all vertices of the given graph. Finally, we get the following Shortest Path Tree (SPT).



Below is the implementation of the above approach:

```
// A Java program for Dijkstra's single source shortest path
// algorithm. The program is for adjacency matrix
// representation of the graph
import java.io.*;
import java.lang.*;
import java.util.*;

class ShortestPath {
	// A utility function to find the vertex with minimum
	// distance value, from the set of vertices not yet
	// included in shortest path tree
	static final int V = 9;
	int minDistance(int dist[], Boolean sptSet[])
	{
		// Initialize min value
		int min = Integer.MAX_VALUE, min_index = -1;

		for (int v = 0; v < V; v++)
			if (sptSet[v] == false && dist[v] <= min) {
				min = dist[v];
				min_index = v;
			}

		return min_index;
	}

	// A utility function to print the constructed distance
	// array
```

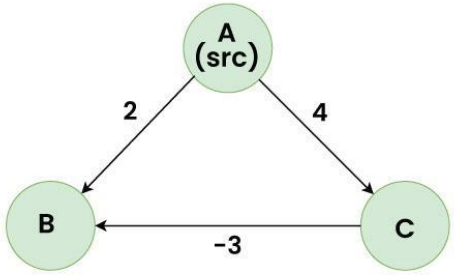
```
void printSolution(int dist[])
{
        System.out.println(
                "Vertex \t\t Distance from Source");
        for (int i = 0; i < V; i++)
                System.out.println(i + " \t\t " + dist[i]);
}

// Function that implements Dijkstra's single source
// shortest path algorithm for a graph represented using
// adjacency matrix representation
void dijkstra(int graph[][], int src)
{
        int dist[] = new int[V]; // The output array.
                                                // dist[i] will hold
        // the shortest distance from src to i

        // sptSet[i] will true if vertex i is included in
        // shortest path tree or shortest distance from src
        // to i is finalized
        Boolean sptSet[] = new Boolean[V];

        // Initialize all distances as INFINITE and stpSet[]
        // as false
        for (int i = 0; i < V; i++) {
                dist[i] = Integer.MAX_VALUE;
                sptSet[i] = false;
        }

        // Distance of source vertex from itself is always 0
        dist[src] = 0;

        // Find shortest path for all vertices
        for (int count = 0; count < V - 1; count++) {
                // Pick the minimum distance vertex from the set
                // of vertices not yet processed. u is always
                // equal to src in first iteration.
                int u = minDistance(dist, sptSet);

                // Mark the picked vertex as processed
                sptSet[u] = true;
```

```
                // Update dist value of the adjacent vertices of
                // the picked vertex.
                for (int v = 0; v < V; v++)

                        // Update dist[v] only if is not in sptSet,
                        // there is an edge from u to v, and total
                        // weight of path from src to v through u is
                        // smaller than current value of dist[v]
                        if (!sptSet[v] && graph[u][v] != 0
                                && dist[u] != Integer.MAX_VALUE
                                && dist[u] + graph[u][v] < dist[v])
                                dist[v] = dist[u] + graph[u][v];
        }

        // print the constructed distance array
        printSolution(dist);
    }

    // Driver's code
    public static void main(String[] args)
    {
        /* Let us create the example graph discussed above
        */
        int graph[][]
                = new int[][] { { 0, 4, 0, 0, 0, 0, 0, 8, 0 },
                                        { 4, 0, 8, 0, 0, 0, 0, 11, 0 },
                                        { 0, 8, 0, 7, 0, 4, 0, 0, 2 },
                                        { 0, 0, 7, 0, 9, 14, 0, 0, 0 },
                                        { 0, 0, 0, 9, 0, 10, 0, 0, 0 },
                                        { 0, 0, 4, 14, 10, 0, 2, 0, 0 },
                                        { 0, 0, 0, 0, 0, 2, 0, 1, 6 },
                                        { 8, 11, 0, 0, 0, 0, 1, 0, 7 },
                                        { 0, 0, 2, 0, 0, 0, 6, 7, 0 } };
        ShortestPath t = new ShortestPath();

        // Function call
        t.dijkstra(graph, 0);
    }
}
```

Why Dijkstra's Algorithms fails for the Graphs having Negative Edges ?

The problem with negative weights arises from the fact that Dijkstra's algorithm assumes that once a node is added to the set of visited nodes, its distance is finalized and will not change. However, in the presence of negative weights, this assumption can lead to incorrect results.

Consider the following graph for the example:



Shortest Distance A to B using Dijkstra = 2
Actual Shortest Distance A to B = 4 - 3 = 1

In the above graph, A is the source node, among the edges A to B and A to C , A to B is the smaller weight and Dijkstra assigns the shortest distance of B as 2, but because of existence of a negative edge from C to B, the actual shortest distance reduces to 1 which Dijkstra fails to detect.

Note: We use Bellman Ford's Shortest path algorithm in case we have negative edges in the graph.

Dijkstra's Algorithm using Adjacency List in O(E logV):

For Dijkstra's algorithm, it is always recommended to use Heap (or priority queue) as the required operations (extract minimum and decrease key) match with the speciality of the heap (or priority queue). However, the problem is, that priority_queue doesn't support the decrease key. To resolve this problem, do not update a key, but insert one more copy of it. So we allow multiple instances of the same vertex in the priority queue. This approach doesn't require decreasing key operations and has below important properties.

- Whenever the distance of a vertex is reduced, we add one more instance of a vertex in priority_queue. Even if there are multiple instances, we only consider the instance with minimum distance and ignore other instances.
- The time complexity remains O(E * LogV) as there will be at most O(E) vertices in the priority queue and O(logE) is the same as O(logV)

Below is the implementation of the above approach:

```
import java.util.*;

class Graph {
	private int V;
	private List<List<iPair>> adj;

	Graph(int V) {
		this.V = V;
		adj = new ArrayList<>();
		for (int i = 0; i < V; i++) {
			adj.add(new ArrayList<>());
		}
	}

	void addEdge(int u, int v, int w) {
		adj.get(u).add(new iPair(v, w));
		adj.get(v).add(new iPair(u, w));
	}

	void shortestPath(int src) {
		PriorityQueue<iPair> pq = new PriorityQueue<>(V, Comparator.comparingInt(o -
> o.first));
		int[] dist = new int[V];
		Arrays.fill(dist, Integer.MAX_VALUE);

		pq.add(new iPair(0, src));
		dist[src] = 0;

		while (!pq.isEmpty()) {
			int u = pq.poll().second;

			for (iPair v : adj.get(u)) {
				if (dist[v.first] > dist[u] + v.second) {
					dist[v.first] = dist[u] + v.second;
					pq.add(new iPair(dist[v.first], v.first));
				}
			}
		}
```

```
            System.out.println("Vertex Distance from Source");
            for (int i = 0; i < V; i++) {
                System.out.println(i + "\t\t" + dist[i]);
            }
        }

        static class iPair {
            int first, second;

            iPair(int first, int second) {
                this.first = first;
                this.second = second;
            }
        }
}

public class Main {
        public static void main(String[] args) {
            int V = 9;
            Graph g = new Graph(V);

            g.addEdge(0, 1, 4);
            g.addEdge(0, 7, 8);
            g.addEdge(1, 2, 8);
            g.addEdge(1, 7, 11);
            g.addEdge(2, 3, 7);
            g.addEdge(2, 8, 2);
            g.addEdge(2, 5, 4);
            g.addEdge(3, 4, 9);
            g.addEdge(3, 5, 14);
            g.addEdge(4, 5, 10);
            g.addEdge(5, 6, 2);
            g.addEdge(6, 7, 1);
            g.addEdge(6, 8, 6);
            g.addEdge(7, 8, 7);

            g.shortestPath(0);
        }
}
```

Worshall's Algorithm-

Worshall's algorithm, also known as the Floyd-Warshall algorithm, is used to find the shortest paths between all pairs of vertices in a weighted graph, including negative-weight edges (but not negative-weight cycles). The algorithm is dynamic programming based and has a time complexity of O(V^3), where V is the number of vertices in the graph.

Here's the Java implementation of the Floyd-Warshall algorithm:

```
import java.util.Arrays;

public class FloydWarshallAlgorithm {
    public static void main(String[] args) {
        int[][] graph = {
                {0, 5, Integer.MAX_VALUE, 10},
                {Integer.MAX_VALUE, 0, 3, Integer.MAX_VALUE},
                {Integer.MAX_VALUE, Integer.MAX_VALUE, 0, 1},
                {Integer.MAX_VALUE, Integer.MAX_VALUE, Integer.MAX_VALUE, 0}
        };

        floydWarshall(graph);

        // Display the resulting matrix with shortest distances
        System.out.println("Shortest Distances between all pairs of vertices:");
        for (int[] row : graph) {
            System.out.println(Arrays.toString(row));
        }
    }

    private static void floydWarshall(int[][] graph) {
        int V = graph.length;

        // Initialization: Initialize the distance matrix with the given graph
        int[][] dist = new int[V][V];
        for (int i = 0; i < V; i++) {
            System.arraycopy(graph[i], 0, dist[i], 0, V);
        }

        // Iterate through all vertices as intermediate vertices
        for (int k = 0; k < V; k++) {
            // Pick all vertices as source one by one
            for (int i = 0; i < V; i++) {
                // Pick all vertices as destination for the above picked source
                for (int j = 0; j < V; j++) {
```

```
                // If vertex k is on the shortest path from i to j,
                // then update the value of dist[i][j]
                if (dist[i][k] != Integer.MAX_VALUE &&
                        dist[k][j] != Integer.MAX_VALUE &&
                        dist[i][k] + dist[k][j] < dist[i][j]) {
                    dist[i][j] = dist[i][k] + dist[k][j];
                }
            }
        }
    }
}
```

Breadth First Search or BFS for a Graph-

Relation between BFS for Graph and Tree traversal:

Breadth-First Traversal (or Search) for a graph is similar to the Breadth-First Traversal of a tree.

The only catch here is, that, unlike trees, graphs may contain cycles, so we may come to the same node again. To avoid processing a node more than once, we divide the vertices into two categories:

- **Visited and**
- **Not visited**.

A boolean visited array is used to mark the visited vertices. For simplicity, it is assumed that all vertices are reachable from the starting vertex. BFS uses a queue data structure for traversal.

How does BFS work?

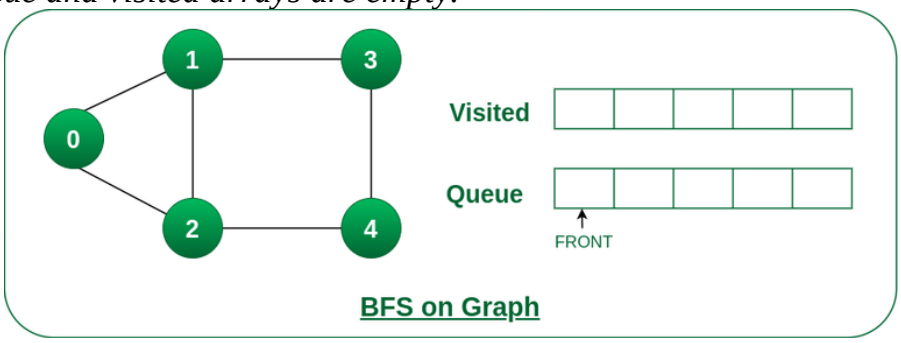
Starting from the root, all the nodes at a particular level are visited first and then the nodes of the next level are traversed till all the nodes are visited.

To do this a queue is used. All the adjacent unvisited nodes of the current level are pushed into the queue and the nodes of the current level are marked visited and popped from the queue.

Illustration:

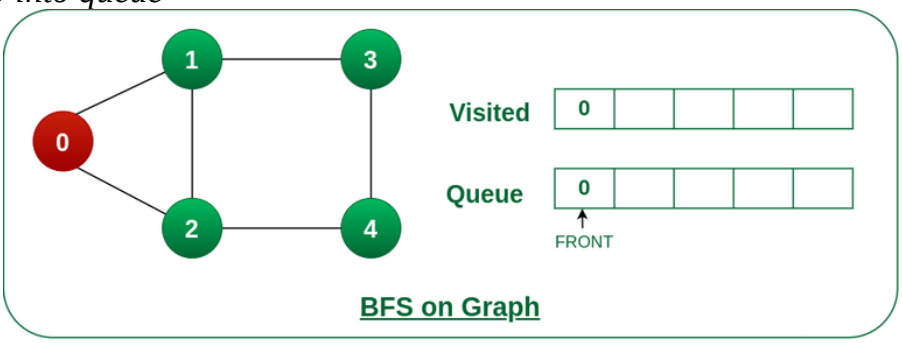
Let us understand the working of the algorithm with the help of the following example.

Step1: *Initially queue and visited arrays are empty.*

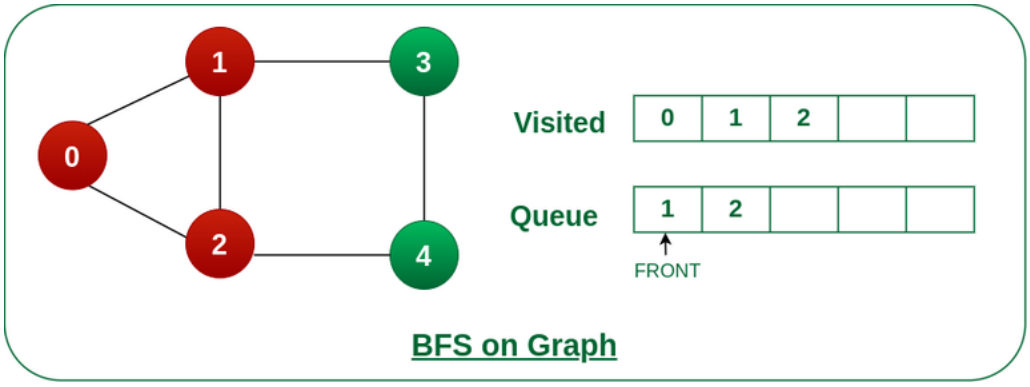


Queue and visited arrays are empty initially.

Step2: *Push node 0 into queue*

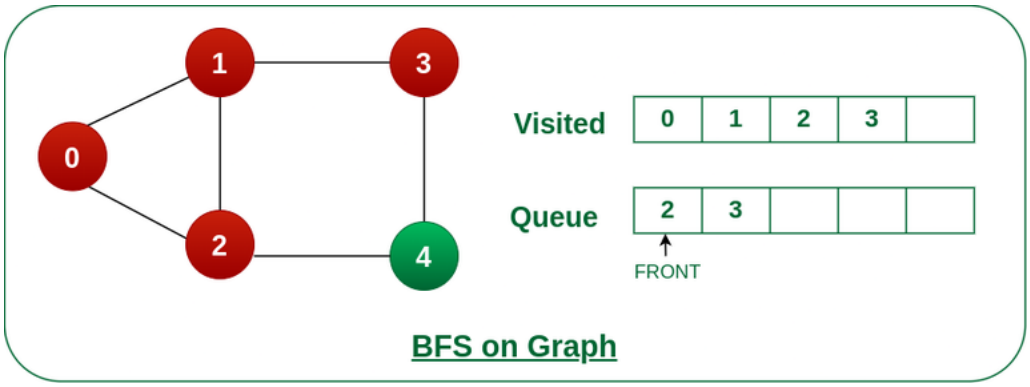


Step 3: *Remove node 0 from the front of queue and visit the unvisited neighbours and push them into queue.*



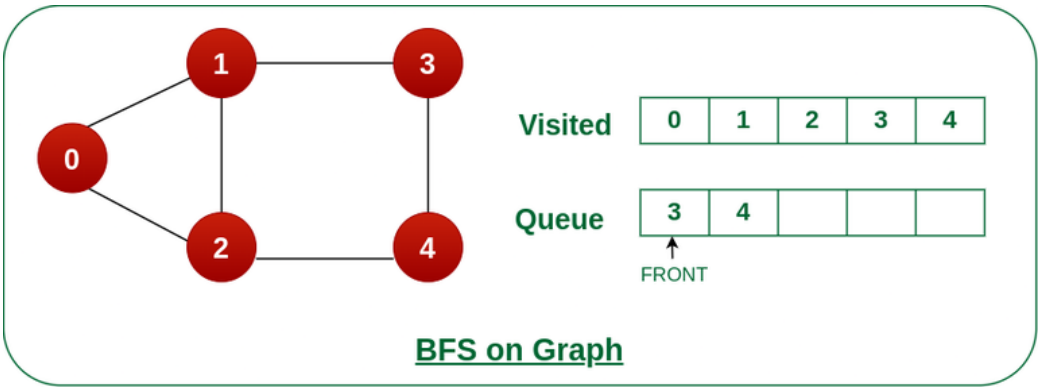
Remove node 0 from the front of queue and visited the unvisited neighbours and push into queue.

Step 4: *Remove node 1 from the front of queue and visit the unvisited neighbours and push them into queue.*



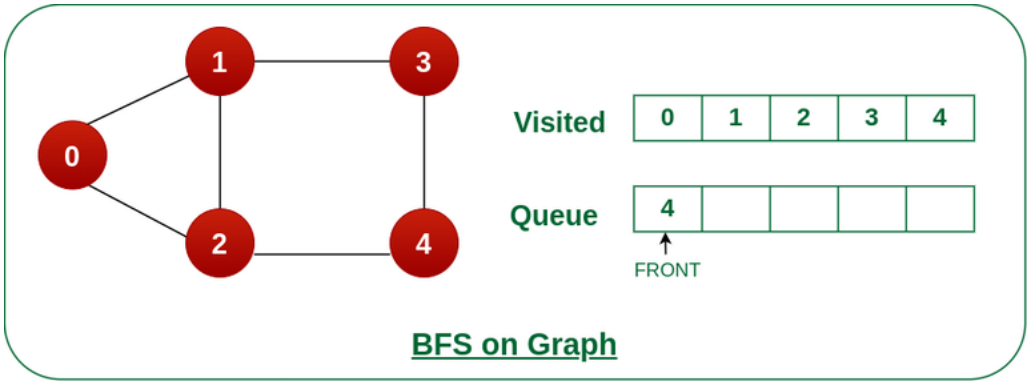
Remove node 1 from the front of queue and visited the unvisited neighbours and push

Step 5: *Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.*



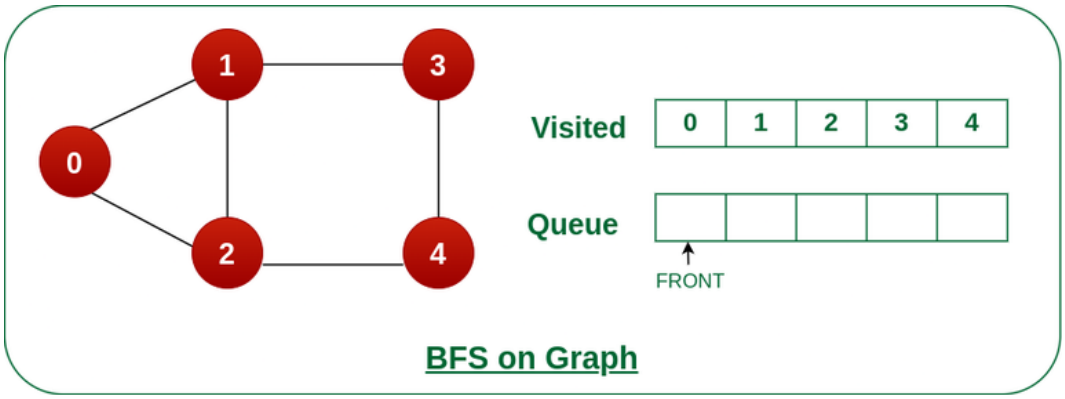
Remove node 2 from the front of queue and visit the unvisited neighbours and push them into queue.

Step 6: *Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.*
As we can see that every neighbours of node 3 is visited, so move to the next node that are in the front of the queue.



Remove node 3 from the front of queue and visit the unvisited neighbours and push them into queue.

Steps 7: *Remove node 4 from the front of queue and visit the unvisited neighbours and push them into queue.*
As we can see that every neighbours of node 4 are visited, so move to the next node that is in the front of the queue.



Remove node 4 from the front of queue and visit the unvisited neighbours and push them into queue.

Now, Queue becomes empty, So, terminate these process of iteration.

Implementation of BFS for Graph using Adjacency List:

```
// Java program to print BFS traversal from a given source
// vertex. BFS(int s) traverses vertices reachable from s.

import java.io.*;
import java.util.*;

// This class represents a directed graph using adjacency
// list representation
class Graph {

	// No. of vertices
	private int V;

	// Adjacency Lists
	private LinkedList<Integer> adj[];

	// Constructor
	Graph(int v)
	{
		V = v;
		adj = new LinkedList[v];
		for (int i = 0; i < v; ++i)
			adj[i] = new LinkedList();
	}

	// Function to add an edge into the graph
	void addEdge(int v, int w) { adj[v].add(w); }

	// prints BFS traversal from a given source s
	void BFS(int s)
	{
		// Mark all the vertices as not visited(By default
		// set as false)
		boolean visited[] = new boolean[V];

		// Create a queue for BFS
		LinkedList<Integer> queue
			= new LinkedList<Integer>();

		// Mark the current node as visited and enqueue it
```

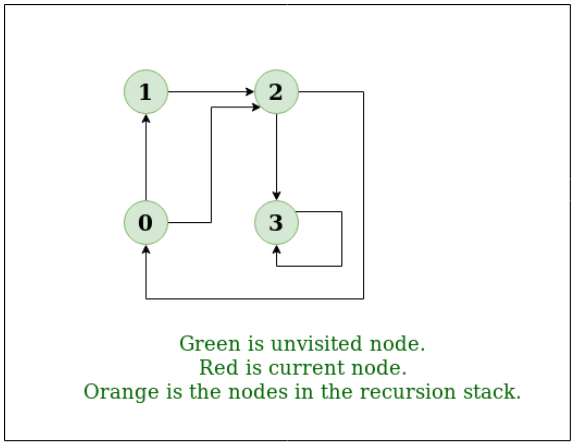
```
        visited[s] = true;
        queue.add(s);

        while (queue.size() != 0) {

            // Dequeue a vertex from queue and print it
            s = queue.poll();
            System.out.print(s + " ");

            // Get all adjacent vertices of the dequeued
            // vertex s.
            // If an adjacent has not been visited,
            // then mark it visited and enqueue it
            Iterator<Integer> i = adj[s].listIterator();
            while (i.hasNext()) {
                int n = i.next();
                if (!visited[n]) {
                    visited[n] = true;
                    queue.add(n);
                }
            }
        }
    }

    // Driver code
    public static void main(String args[])
    {
        Graph g = new Graph(4);
        g.addEdge(0, 1);
        g.addEdge(0, 2);
        g.addEdge(1, 2);
        g.addEdge(2, 0);
        g.addEdge(2, 3);
        g.addEdge(3, 3);

        System.out.println(
            "Following is Breadth First Traversal "
            + "(starting from vertex 2)");

        g.BFS(2);
    } }
```

Depth First Search or DFS for a Graph-

Depth First Traversal (or DFS) for a graph is similar to Depth First Traversal of a tree. The only catch here is, that, unlike trees, graphs may contain cycles (a node may be visited twice). To avoid processing a node more than once, use a boolean visited array. A graph can have more than one DFS traversal.

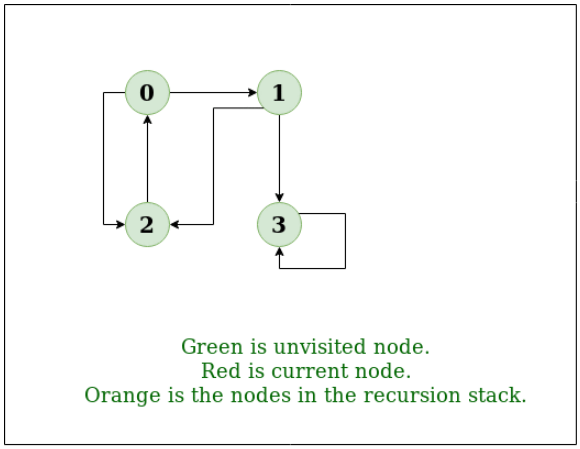
Example:

Input: *n = 4, e = 6*
0 -> 1, 0 -> 2, 1 -> 2, 2 -> 0, 2 -> 3, 3 -> 3
Output: *DFS from vertex 1 : 1 2 0 3*
Explanation:
DFS Diagram:



Example 1

Input: *n = 4, e = 6*
2 -> 0, 0 -> 2, 1 -> 2, 0 -> 1, 3 -> 3, 1 -> 3
Output: *DFS from vertex 2 : 2 0 1 3*
Explanation:
DFS Diagram:



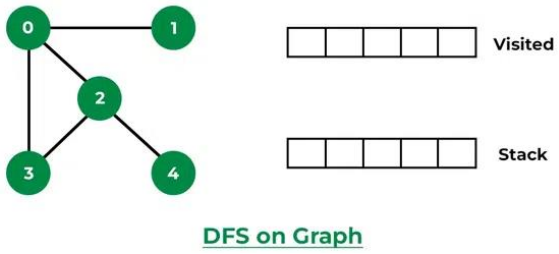
Example 2

How does DFS work?

Depth-first search is an algorithm for traversing or searching tree or graph data structures. The algorithm starts at the root node (selecting some arbitrary node as the root node in the case of a graph) and explores as far as possible along each branch before backtracking.

Let us understand the working of Depth First Search with the help of the following illustration:

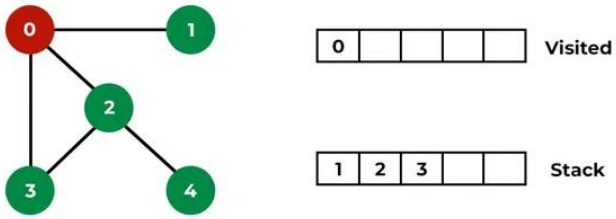
Step1: *Initially stack and visited arrays are empty.*



DFS on Graph

Stack and visited arrays are empty initially.

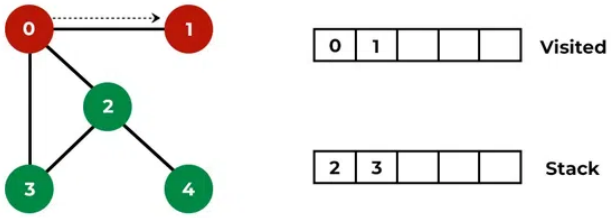
Step 2: *Visit 0 and put its adjacent nodes which are not visited yet into the stack.*



DFS on Graph

Visit node 0 and put its adjacent nodes (1, 2, 3) into the stack

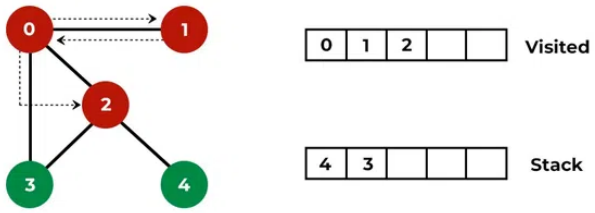
Step 3: *Now, Node 1 at the top of the stack, so visit node 1 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.*



DFS on Graph

Visit node 1

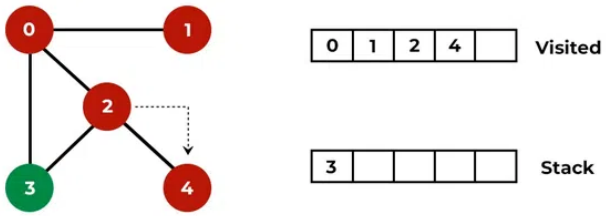
Step 4: *Now, Node 2 at the top of the stack, so visit node 2 and pop it from the stack and put all of its adjacent nodes which are not visited (i.e, 3, 4) in the stack.*



DFS on Graph

Visit node 2 and put its unvisited adjacent nodes (3, 4) into the stack

Step 5: *Now, Node 4 at the top of the stack, so visit node 4 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.*



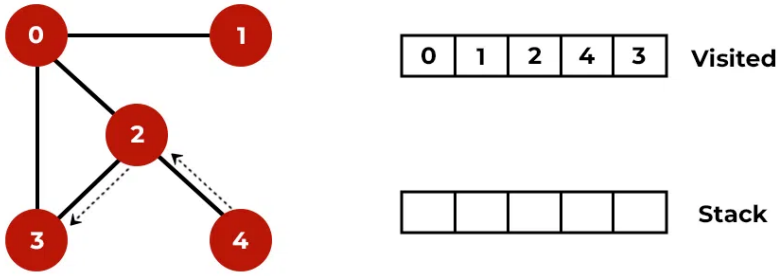
DFS on Graph

Visit node 4

Step 6: *Now, Node 3 at the top of the stack, so visit node 3 and pop it from the stack and put all of its adjacent nodes which are not visited in the stack.*

Visit node 3

Now, Stack becomes empty, which means we have visited all the nodes and our DFS traversal ends.



DFS on Graph

Depth First Search or BFS for a Graph-

```
import java.util.*;
class Graph {
    private int vertices;
    private Map<Integer, List<Integer>> adjacencyList;

    public Graph(int vertices) {
        this.vertices = vertices;
        this.adjacencyList = new HashMap<>();

        for (int i = 0; i < vertices; i++) {
            adjacencyList.put(i, new ArrayList<>());
        }
    }

    public void addEdge(int source, int destination) {
        adjacencyList.get(source).add(destination);
        adjacencyList.get(destination).add(source); // For undirected graph
    }
    public void dfs(int startVertex) {
        Set<Integer> visited = new HashSet<>();
        dfsRecursive(startVertex, visited);
    }

    private void dfsRecursive(int currentVertex, Set<Integer> visited) {
        visited.add(currentVertex);
        System.out.print(currentVertex + " ");

        for (int neighbor : adjacencyList.get(currentVertex)) {
            if (!visited.contains(neighbor)) {
                dfsRecursive(neighbor, visited);
            }
        }
    }
}

public class DFSExample {
    public static void main(String[] args) {
        Graph graph = new Graph(6);

        // Adding edges
```

```
        graph.addEdge(0, 1);
        graph.addEdge(0, 2);
        graph.addEdge(1, 3);
        graph.addEdge(2, 4);
        graph.addEdge(3, 5);

        System.out.println("Depth-First Search starting from vertex 0:");
        graph.dfs(0);
    }
}
```

Directed acyclic Graphs-

A Directed Acyclic Graph (DAG) is a graph that is directed and contains no cycles. In other words, it is a directed graph with no directed cycles. DAGs have important applications in various domains, such as scheduling, task dependencies, and representing certain kinds of relationships.

Here's a simple Java implementation of a directed acyclic graph, along with some basic operations:

```
import java.util.*;
class DirectedAcyclicGraph {
    private Map<Integer, List<Integer>> adjacencyList;

    public DirectedAcyclicGraph() {
        this.adjacencyList = new HashMap<>();
    }

    public void addVertex(int vertex) {
        adjacencyList.put(vertex, new ArrayList<>());
    }

    public void addEdge(int source, int destination) {
        if (isCyclicWithEdge(source, destination)) {
            System.out.println("Warning: Adding this edge would create a cycle. Edge not added.");
            return;
        }
        adjacencyList.get(source).add(destination);
    }

    private boolean isCyclicWithEdge(int source, int destination) {
        Set<Integer> visited = new HashSet<>();
        Stack<Integer> stack = new Stack<>();

        stack.push(source);

        while (!stack.isEmpty()) {
            int current = stack.pop();

            if (visited.contains(current)) {
                return true; // Cycle detected
            }

            visited.add(current);
```

```
            for (int neighbor : adjacencyList.get(current)) {
                stack.push(neighbor);
            }
        }

        // Check if adding the edge creates a cycle
        visited.clear();
        Stack<Integer> stack2 = new Stack<>();
        stack2.push(destination);

        while (!stack2.isEmpty()) {
            int current = stack2.pop();

            if (visited.contains(current)) {
                return true; // Cycle detected
            }

            visited.add(current);

            for (int neighbor : adjacencyList.get(current)) {
                stack2.push(neighbor);
            }
        }

        return false; // No cycle detected
    }

    public void displayGraph() {
        for (int vertex : adjacencyList.keySet()) {
            System.out.print("Vertex " + vertex + " -> ");
            for (int neighbor : adjacencyList.get(vertex)) {
                System.out.print(neighbor + " ");
            }
            System.out.println();
        }
    }
}
```

```
public class DirectedAcyclicGraphExample {
    public static void main(String[] args) {

        DirectedAcyclicGraph dag = new DirectedAcyclicGraph();

        dag.addVertex(1);
        dag.addVertex(2);
        dag.addVertex(3);
        dag.addVertex(4);

        dag.addEdge(1, 2);
        dag.addEdge(1, 3);
        dag.addEdge(2, 3);
        dag.addEdge(3, 4);

        // Adding an edge that would create a cycle (2 -> 1)
        dag.addEdge(2, 1);

        System.out.println("Directed Acyclic Graph:");
        dag.displayGraph();
    }
}
```

Minimal spanning trees- (Kruskal's Algorithm)

A Minimal Spanning Tree (MST) is a subset of the edges of a connected, undirected graph that connects all the vertices together without any cycles and with the minimum possible total edge weight. Two classic algorithms for finding the minimal spanning tree of a graph are Kruskal's algorithm and Prim's algorithm.

Here's an example of a Java implementation of Kruskal's algorithm for finding the Minimal Spanning Tree:

```
import java.util.*;
class Edge implements Comparable<Edge> {
    int source, destination, weight;

    public Edge(int source, int destination, int weight) {
        this.source = source;
        this.destination = destination;
        this.weight = weight;
    }

    @Override
    public int compareTo(Edge other) {
        return Integer.compare(this.weight, other.weight);
    }
}

class KruskalMST {
    private List<Edge> edges;
    private int[] parent;

    public KruskalMST(int numberOfVertices) {
        edges = new ArrayList<>();
        parent = new int[numberOfVertices];

        for (int i = 0; i < numberOfVertices; i++) {
            parent[i] = i; // Initialize each vertex as its own parent
        }
    }

    public void addEdge(int source, int destination, int weight) {
        edges.add(new Edge(source, destination, weight));
    }
```

```
    public List<Edge> findMST() {
        List<Edge> minimumSpanningTree = new ArrayList<>();
        Collections.sort(edges); // Sort edges by weight

        for (Edge edge : edges) {
            int sourceRoot = findRoot(edge.source);
            int destinationRoot = findRoot(edge.destination);

            if (sourceRoot != destinationRoot) {
                // Adding this edge does not form a cycle, include it in the MST
                minimumSpanningTree.add(edge);
                union(sourceRoot, destinationRoot);
            }
        }

        return minimumSpanningTree;
    }

    private int findRoot(int vertex) {
        while (parent[vertex] != vertex) {
            // Path compression: set each vertex's parent to its grandparent
            parent[vertex] = parent[parent[vertex]];
            vertex = parent[vertex];
        }
        return vertex;
    }

    private void union(int root1, int root2) {
        parent[root1] = root2;
    }
}

public class KruskalMSTExample {
    public static void main(String[] args) {
        KruskalMST kruskalMST = new KruskalMST(6);

        // Adding edges with weights
        kruskalMST.addEdge(0, 1, 5);
        kruskalMST.addEdge(0, 2, 3);
        kruskalMST.addEdge(1, 2, 1);
        kruskalMST.addEdge(1, 3, 4);
```

```
        kruskalMST.addEdge(2, 3, 2);
        kruskalMST.addEdge(2, 4, 7);
        kruskalMST.addEdge(3, 4, 6);

        List<Edge> minimumSpanningTree = kruskalMST.findMST();

        System.out.println("Edges in the Minimal Spanning Tree:");
        for (Edge edge : minimumSpanningTree) {
            System.out.println(edge.source + " -- " + edge.destination + " Weight: " + edge.weight);
        }
    }
}
```

Minimal spanning trees- (Prims Algorithm)

Prim's algorithm is another widely used algorithm to find the Minimal Spanning Tree (MST) of a connected, undirected graph. It starts with an arbitrary vertex and grows the MST by always selecting the edge with the smallest weight that connects a vertex in the MST to a vertex outside the MST.

Here's an example of a Java implementation of Prim's algorithm:

```
import java.util.*;
class PrimMST {
    private int numberOfVertices;
    private List<List<Edge>> adjacencyList;

    public PrimMST(int numberOfVertices) {
        this.numberOfVertices = numberOfVertices;
        this.adjacencyList = new ArrayList<>();

        for (int i = 0; i < numberOfVertices; i++) {
            adjacencyList.add(new ArrayList<>());
        }
    }

    public void addEdge(int source, int destination, int weight) {
        adjacencyList.get(source).add(new Edge(source, destination, weight));
        adjacencyList.get(destination).add(new Edge(destination, source, weight)); // For
undirected graph
    }

    public List<Edge> findMST() {
        List<Edge> minimumSpanningTree = new ArrayList<>();
        PriorityQueue<Edge> priorityQueue = new
PriorityQueue<>(Comparator.comparingInt(edge -> edge.weight));
        Set<Integer> visited = new HashSet<>();

        // Start from vertex 0 (can start from any vertex)
        int startVertex = 0;
        visited.add(startVertex);

        while (visited.size() < numberOfVertices) {
            for (Edge edge : adjacencyList.get(startVertex)) {
                if (!visited.contains(edge.destination)) {
                    priorityQueue.offer(edge);
                }
```

```
            }

            Edge minEdge = priorityQueue.poll();

            if (visited.contains(minEdge.destination)) {
                continue;
            }

            minimumSpanningTree.add(minEdge);
            visited.add(minEdge.destination);
            startVertex = minEdge.destination;
        }

        return minimumSpanningTree;
    }
}

public class PrimMSTExample {
    public static void main(String[] args) {
        PrimMST primMST = new PrimMST(6);

        // Adding edges with weights
        primMST.addEdge(0, 1, 5);
        primMST.addEdge(0, 2, 3);
        primMST.addEdge(1, 2, 1);
        primMST.addEdge(1, 3, 4);
        primMST.addEdge(2, 3, 2);
        primMST.addEdge(2, 4, 7);
        primMST.addEdge(3, 4, 6);

        List<Edge> minimumSpanningTree = primMST.findMST();

        System.out.println("Edges in the Minimal Spanning Tree:");
        for (Edge edge : minimumSpanningTree) {
            System.out.println(edge.source + " -- " + edge.destination + " Weight: " + edge.weight);
        }
    }
}
```

```
class Edge {
    int source, destination, weight;

    public Edge(int source, int destination, int weight) {
        this.source = source;
        this.destination = destination;
        this.weight = weight;
    }
}
```

Application to the travelling salesman problem-

The Traveling Salesman Problem (TSP) is a classic optimization problem in which the goal is to find the shortest possible route that visits a given set of cities and returns to the starting city. The problem is NP-hard, meaning that there is no known polynomial-time solution for large instances.

While Prim's and Kruskal's algorithms find the Minimal Spanning Tree (MST) in a graph, they don't directly solve the TSP. However, the MST can be useful in the context of the TSP. One common approach is to use the MST to construct a tour by traversing each edge twice.

Here's a simple example of how you might use Prim's algorithm to approximate a solution for the TSP:

1. Use Prim's algorithm to find the Minimal Spanning Tree (MST) of the given graph.
2. Traverse the MST in a depth-first manner, and record the order in which vertices are visited.
3. The resulting tour will start and end at the root of the MST, forming a cycle.

Let's modify the PrimMST class from the previous example to include a method for constructing a tour:

```
import java.util.*;

class PrimMST {
   // ... (unchanged code)

   public List<Integer> constructTour() {
      List<Integer> tour = new ArrayList<>();
      Set<Integer> visited = new HashSet<>();

      // Start from vertex 0 (can start from any vertex)
      int startVertex = 0;
      visited.add(startVertex);

      constructTourDFS(startVertex, visited, tour);

      return tour;
   }
   private void constructTourDFS(int currentVertex, Set<Integer> visited, List<Integer> tour) {
      tour.add(currentVertex);

      for (Edge edge : adjacencyList.get(currentVertex)) {
         if (!visited.contains(edge.destination)) {
            visited.add(edge.destination);
            constructTourDFS(edge.destination, visited, tour);
         }
```

```
        }
    }
}

public class TSPWithPrimMST {
    public static void main(String[] args) {
        PrimMST primMST = new PrimMST(6);

        // Adding edges with weights
        primMST.addEdge(0, 1, 5);
        primMST.addEdge(0, 2, 3);
        primMST.addEdge(1, 2, 1);
        primMST.addEdge(1, 3, 4);
        primMST.addEdge(2, 3, 2);
        primMST.addEdge(2, 4, 7);
        primMST.addEdge(3, 4, 6);

        List<Integer> tour = primMST.constructTour();

        System.out.println("Approximate TSP Tour:");
        System.out.println(tour);
    }
}

class Edge {
    // ... (unchanged code)
}
```